

MaLAREa SG1 - Machine Learner for Automated Reasoning with Semantic Guidance

Josef Urban^{1,*}, Geoff Sutcliffe², Petr Pudlák¹, and Jiří Vyskočil¹

¹ Charles University, Czech Republic

² University of Miami, USA

Abstract. This paper describes a system combining model-based and learning-based methods for automated reasoning in large theories, i.e. on a large number of problems that use many axioms, lemmas, theorems, definitions, and symbols, in a consistent fashion. The implementation is based on the existing MaLAREa system, which cycles between theorem proving attempts and learning axiom relevance from successes. This system is extended by taking into account semantic relevance of axioms, in a way similar to that of the SRASS system. The resulting combined system significantly outperforms both MaLAREa and SRASS on the MPTP Challenge large theory benchmark, in terms of both the number of problems solved and the time taken to find solutions. The design, implementation, and experimental testing of the system are described here.

1 Introduction

In recent years the ability of systems to reason over large theories – theories in which there are many functors and predicates, many axioms of which typically only a few are required for the proof of a theorem, and many theorems to be proved from the same set of axioms – has become more important. Large theory problems are becoming more prevalent as large knowledge bases, e.g., ontologies and large mathematical knowledge bases, are translated into forms suitable for automated reasoning [12,13,22], and mechanical generation of automated reasoning problems becomes more common, e.g., [4,8]. Over the years there have been regular investigations into techniques for the a priori selection of the necessary axioms, e.g., [9,17,7,21,11], and the use of externally provided lemmas, e.g., [24,5,26,21,10]. Performances on large theory problems in the TPTP library [19] and the associated CADE ATP System Competition [16] show that automated reasoning systems have improved their ability to select and use the necessary axioms.

The work described in this paper addresses the issue of selecting necessary axioms, from a large set also containing superfluous axioms, to obtain a proof of a conjecture. It combines ordering of axioms by machine learning their relevance to features of conjectures, as found in the MaLAREa system [23], with use of

* Supported by a Marie Curie International Fellowship within the 6th European Community Framework Programme.

semantic guidance to advise on the selection of axioms, as found in the SRASS system [18]. The resulting system significantly improves the performance of the two above mentioned systems on the MPTP Challenge¹ large theory benchmark, both in terms of the number of problems solved, and in the speed of processing.

This paper is organized as follows: Section 2 describes the machine learning approach. Section 3 describes the semantic selection technique. Section 4 explains how these two have been combined. Section 5 provides a detailed example of how the combined system works. Section 6 mentions some other improvements that are relevant to the performance of the system. Section 7 gives test results, with commentary. Section 8 concludes and discusses possible future research.

2 Machine Learning Axiom Relevance

The basic idea of the machine learning approach is to interleave ATP system runs on many related problems with learning from successful proofs, and to use the learned knowledge for limiting the set of axioms given to the ATP systems in the following runs. In general, the goal is to learn a mapping from features (in machine learning terminology) of the conjectures (or even of the whole problems when speaking generally) to proving methods that are successful when the features are present. This general setting was instantiated in the first MaLAREa version in the following way: The features characterizing conjectures are the symbols appearing in them, and the proving method is ordering of axioms according to their expected relevance to proving the conjecture.

The SNoW machine learning system [2] is used by MaLAREa in naive Bayes mode, producing an updated Bayes net after each run, linking symbols with the axioms that were useful for proving conjectures containing those symbols. For learning and evaluation in SNoW, all symbols and axiom names are disjointly translated to integers. The integers corresponding to symbols are the input features, and those corresponding to axioms are the output features. As ATP backends the E prover [15] and SPASS [25] are used by default.

MaLAREa is highly configurable, but the default algorithm is as follows: Minimal and maximal number of axioms and time limits are given (4 and 256 axioms, and 1 and 64 seconds by default) – these ranges have been established through experience with the MPTP Challenge problems. First, all ATP backends (E and SPASS by default) are run on all problems (an ATP *problem* is the conjecture and a selection of axioms), using all the axioms and the maximal time limit, to produce as much base knowledge as possible. Second, SNoW is trained on examples saying that each axiom is useful for proving itself, i.e., an example created from an axiom A contains the symbols appearing in A as input features, and A itself as the output feature. This Bayes net is evaluated on each conjecture's symbols, producing an ordering of axioms for each conjecture, according to their symbol overlap with the conjecture. For each conjecture its axiom ordering is used to select the maximal number of axioms, and the resultant problem is given to the ATP systems one-at-a-time, with the minimal time limit. If any

¹ <http://www.tptp.org/MPTPChallenge/>

ATP system returns a countersatisfiable result², then no further proof attempts are made on the problem. Successful proofs are turned into training examples containing the conjectures' symbols as input features, and the axioms used in the corresponding proofs as output features. These training examples are added to the previous examples, and a new Bayes net is trained from them all. The evaluation of the Bayes net on unproved conjectures, selection of axioms, ATP proof attempts, and retraining, iterates. The number of axioms selected and time limit used is adapted according to whether or not any proofs are found in each iteration. When a proof is found in a pass, the axiom and time limits drop to their minimal values for the next pass (hoping that the relevance update caused by the new solution will make some more problems easy to solve). If no proofs are found in a pass, first the axiom limit is doubled (hoping that considering more axioms will help) until the maximal axiom limit is reached. After that the time limit is quadrupled, hoping that "harder thinking" will help, until the maximal time limit is reached. The algorithm stops when the maximal axiom and time limits are reached.

One (default) change to this basic algorithm is a countersatisfiability precheck, to avoid axiom set selections that do not make sense in the light of previous results. If a new selection of axioms for a countersatisfiable problem's conjecture is a subset of the countersatisfiable problem's axioms, it makes no sense to form a new problem with the new selection of axioms. In this situation, if the time limit is sufficiently low (1s by default), axioms are added in order of their relevance until the new set is no longer a subset of any set which produced a countersatisfiable result for the conjecture. In such cases the axiom limit is "soft", and may be violated.

3 Semantic Relevance Axiom Selection

The semantic relevance selection algorithm implemented in SRASS selects axioms for each conjecture independently, with no benefit accrued from previous successful selections. For each problem it starts with an empty set of selected axioms. At each pass the process looks for a model of the selected axioms and the negation of the conjecture. If no such model exists then the conjecture is a logical consequence of the selected axioms. If such a model exists then an unselected axiom that is false in the model is moved to the set of selected axioms. The newly selected axiom excludes the model (and possibly other models) from the set of models of the selected axioms and negated conjecture, eventually leading to the situation where there are no models of the selected axioms and the negated conjecture. The Figure 1 shows the idea. The plane represents the space of interpretations, the rectangle encompasses the models of the conjecture C , and an oval encompasses the models of the corresponding axiom A_i . In the first

² Status values, such as countersatisfiable, are taken from the SZS ontology [20]. The ontology provides result and output forms that specify what has been established about an ATP problem. For example, countersatisfiable problems have a non-provable conjecture.

pass, when the set of selected axioms is empty, the model M_0 of the negation of the conjecture, $\neg C$, is found. That leads to the selection of the axiom A_1 , which is false in the model. Iteratively, the model M_1 of $\{A_1, \neg C\}$ is found, leading to the selection of A_2 , the model M_2 of $\{A_1, A_2, \neg C\}$ is found, leading to the selection of A_3 , at which point there is no model of $\{A_1, A_2, A_3, \neg C\}$, proving that C is a logical consequence of $\{A_1, A_2, A_3\}$. In the last part of the figure this is seen by the intersection of the axiom ovals lying within the conjecture rectangle.

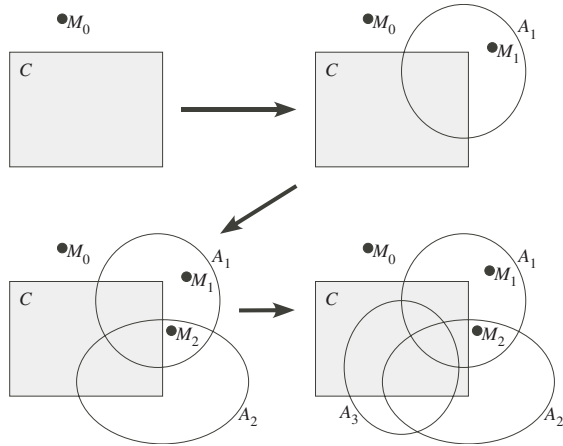


Fig. 1. The Basic Process

Example: Consider the simple propositional problem, to prove the conjecture $C = b$ from the axioms $E_1 = a \mid b$, $E_2 = b \Rightarrow a$, $E_3 = (\neg a \ \& \ (b \mid c)) \mid (a \ \& \ \neg b \ \& \ \neg c)$. and $E_4 = b \mid (a \Leftrightarrow c)$. The conjecture C can be proved from E_3 and E_4 , i.e., E_1 and E_2 are superfluous. The following table shows a possible sequence of models and selected axioms. Note how E_1 , E_2 , and E_3 are true in the first model, so only E_4 can be selected. E_1 and E_3 are false in the second model, but E_1 is found first. E_2 is true in every model of $\neg C$, and thus can never be selected. If the model $\{a, \neg b, c\}$ had been used in the second pass, then E_3 would have been selected, leading to immediate success.

Selected set	Model	Axiom
1 $\{\}$	$\{a, \neg b, \neg c\}$	$E_4 = b \mid (a \Leftrightarrow c)$
2 $\{E_4\}$	$\{\neg a, \neg b, \neg c\}$	$E_1 = a \mid b$
3 $\{E_1, E_4\}$	$\{a, \neg b, c\}$	$E_3 = (\neg a \ \& \ (b \mid c)) \mid (a \ \& \ \neg b \ \& \ \neg c)$
4 $\{E_1, E_3, E_4\}$	-	-

The order in which the axioms are considered at each pass of this process is computed in at the start, according to the *contextual indirect relevance* of each axiom to the conjecture. for the axiom to contribute to a proof of a conjecture F_c , in the context of a set S of axioms and the conjecture. First, the contextual

direct relevance between all formulae pairs $F_1, F_2 \in Axioms \cup \{Conjecture\}$ is measured by

$$\frac{\sum_{s \in (sym(F_1) \cap sym(F_2))} \left(1 - \frac{|\{f: f \in S, s \in sym(f)\}|}{|S|}\right)}{|sym(F_1) \cup sym(F_2)|}$$

where $sym(F)$ is the set of function and predicate symbols occurring in F . Next, the contextual path relevance of every path $F_a = F_1 \cdot F_2 \cdot \dots \cdot F_n = F_c$ from an axiom F_a to the conjecture F_c is calculated as the smallest contextual direct relevance in the path, divided by the length of the path. Finally, the contextual indirect relevance between F_a and F_c is taken as the maximal contextual path relevance over all paths connecting F_a to F_c .

In the SRASS implementation a range of further extensions are used to improve performance, but these are not relevant here.

4 Combining Machine Learning with Semantic Selection

The addition of semantic selection to MaLAREa consists of three extensions to the selection of axioms based on relevance. To implement the extensions a good model finder, Paradox [3], is added to the set of ATP backends.

The first extension is very simple: it checks for countersatisfiability in runs where this is probable – currently when the time limit is 1s and at most 64 axioms are selected. Paradox can detect countersatisfiability much more effectively than E and SPASS, especially if there are very few axioms, and thus eliminates more hopeless proof attempts. This in turn allows the countersatisfiability precheck to detect more cases when more axioms need to be added.

The second extension uses the models found when a problem is found to be countersatisfiable, as an additional criterion for computing axiom relevance. For this extension it is necessary to efficiently evaluate formulae in the models. This (a bit surprisingly) turns out to be a problem both with the Paradox and Darwin [1] systems. The only existing tool for this task that we know of is Bill McCune’s `clausefilter` program, which comes with the Mace4 [6] model finder. Unfortunately, the model formats used by Paradox and Mace4 are incompatible. Paradox is more successful than Mace4 (one of the reasons is that Mace4 avoids models of cardinality 1), and so more useful for the first extension. Therefore, Mace4 is run after Paradox, and only when Paradox finds a model. If Mace4 finds a model, the model is stored, and the truth values of all axioms and conjectures are computed wrt the model by `clausefilter`. This is very fast, and poses no efficiency problems. Often some of the axioms and conjectures cannot be evaluated in the model, because they contain symbols not interpreted by the model. The values of such formulae are therefore undefined wrt the model. For each model, the set of symbols it interprets, the set of true formulae, and the set of false formulae, are recorded. The models are serially numbered, and a hash value (just a SHA1 (160-bit) checksum) is recorded to quickly detect duplicate models. For each formula a vector of its models and a vector of its countermodels (models in which it is false) are recorded.

The stored models are used following the axiom selection approach described in Section 3, i.e., excluding models of the axioms and negated conjecture by adding axioms that are false in the models. Given a conjecture C , if an axiom is false in a model of $\neg C$ then the axiom excludes that model. The more such models an axiom excludes, the more it is likely to be useful. Also, if only a few axioms exclude a model, those axioms are more likely to be useful. This kind of model-based relevance is implemented in the same way as the symbol-based relevance. For each axiom and conjecture a training example is produced. The example has the countermodels (again encoded as integers) of the formula as input features, and the formula itself as the output feature. These examples are added to those described in Section 2. The resulting Bayes net is then evaluated on the features of unproved conjectures, i.e., the symbols and countermodels of the conjecture. This leads to increased relevance of axioms that are false in many (and more rare) countermodels of the conjecture. The relevance boost caused by the model information can be quite big, see, e.g., the promotion of axiom `t143_relat_1` in Section 5.

The third extension goes one step closer to the model-based algorithm described in Section 3. It complements the second extension in way that is analogous to the way that the subset precheck complements the machine learned selection of axioms – by extending the axiom specification using a “logical” criterion. This criterion is as follows: the set of axioms should exclude as many known models of the negated conjecture as possible. While the second extension only promoted the relevance of axioms that contribute to this goal, this extension attempts to meet this goal as much as possible. Given a conjecture C and an axiom selection A , the set NM of the models of $\neg C$ is retrieved. Models already excluded by axioms in A (countermodels of axioms in A) are deleted from NM . If some models of $\neg C$ remain after this, the remaining axioms are inspected in their relevance order, and included into the specification if they exclude additional models of $\neg C$. This continues until either all models of $\neg C$ are excluded by the axioms, or there are no more axioms available. The complete algorithm, combining the axiom relevance and semantic selection, is shown in Figure 2. See Section 5 for a detailed example of how this algorithm proceeds, and particularly Section 5.1 for discussion of some choices in its implementation.

The semantic selection in **MaLAREa** has several significant differences to that of **SRASS**. While syntactic relevance ordering of axioms is used there in the beginning, it remains static. Here the relevance ordering is continuously updated, taking into account successful proofs and model-based relevance. The models created in all proof attempts here are immediately used for evaluation of all axioms and conjectures, and this information is re-used many times in many problems. Multiple axioms can be selected to extend the specifications by the model reasoner in one pass, because the whole database of models is taken into account. An important byproduct of the algorithm is the evolving database of interesting and relevant models. Its final size for the experiments conducted in Section 7 is ca. 1000 models. The models are interesting and relevant because they are products of proof attempts based on the “world knowledge”

accumulated in previous passes. To a large extent the model reasoner prevents creation of the same model twice in two different passes: new models typically have to have a new characteristic vector of truth values on all formulae. One possible future use of the model database is for semantic detection of interesting lemmas and conjectures: a lemma or conjecture is interesting when its characteristic vector on the database of models differs from others.

5 Detailed Example

The methods described in Section 4 are illustrated on the proof of the theorem `t144_relat_1`, during the system's processing of the MPTP Challenge problems. The theorem says that an image of a set under a relation is a subset of the relation's range. In TPTP and Mizar notations it is as follows:

```
! [A,B] : (relation(B) => subset(relation_image(B,A), relation_rng(B)))
for X being set for R being Relation holds R .: X c= rng R
```

The axioms named below are available in TPTP form from the MPTP Challenge distribution, and in Mizar form from Mizar proofs of the MPTP Challenge problems.³ This theorem was proved in the fifth 1 second pass of the system. In the first pass all 198 axioms were considered, while in the remaining four passes the limit of four axioms was used. In the end the theorem was proved from the axioms `d3_tarski`, `t143_relat_1`, `t20_relat_1`.

The axioms used in the second to fifth passes were:

```
2: t25_relat_1,t99_relat_1,t118_relat_1,t45_relat_1
3: t99_relat_1,t88_relat_1,t20_relat_1,t45_relat_1,t116_relat_1,t44_relat_1
4: t20_relat_1,t99_relat_1,t88_relat_1,t45_relat_1,d3_tarski,t116_relat_1,
   t44_relat_1
5: t20_relat_1,d3_tarski,t99_relat_1,t143_relat_1,t45_relat_1,
   reflexivity_r1_tarski
```

In pass 2 the soft limit of four axioms was observed, while in the other three the limit was violated – two or three extra axioms were added by the subset precheck and by model reasoning to the four axioms selected by relevance. In pass 3 the four axioms `t99_relat_1`, `t88_relat_1`, `t20_relat_1`, `t45_relat_1` were selected initially, by their relevance from previous successful proofs and by their symbolic and structural similarity with the conjecture. The resultant problem was not known to be countersatisfiable, so there was no need to add axioms for that reason. However in pass 2 the following four models of the negated conjecture `t144_relat_1` had been found (we only give the model numbers, conjecture that lead to their discovery, and formulae that were evaluated as false in them by `clausefilter`):

³ http://lipa.ms.mff.cuni.cz/~urban/xmlmml/html_abstr.930/00mptp_chall_topdown.html

Input: *MinAxioms*, *MaxAxioms*, *MinCPU*, *MaxCPU*, *CSACPULimit*,
Conjectures, *AllAxioms*, for each $C \in \text{Conjectures}$ $\text{Axioms}(C) \subseteq \text{AllAxioms}$

```

1 TrainingSet =  $\emptyset$ , NumAxioms = MaxAxioms, CPULimit = MinCPU;
2 forall  $F \in (\text{AllAxioms} \cup \text{Conjectures})$  do
3    $\text{CMOSet}(F) = \emptyset$ ; // The CounterModels of  $F$ 
4   TrainingSet = TrainingSet  $\cup \{\langle \text{Symbols}(F), F \rangle\}$ ;
5 end
6 forall  $C \in \text{Conjectures}$  do
7    $\text{CSASet}(C) = \emptyset$ ; // The CounterSatisfiable axiom sets for  $C$ 
8   Result = Run systems on  $\text{Axioms}(C) \models C$  with MaxCPU;
9   ProcessResult(C, Result,  $\text{Axioms}(C)$ );
10 end
11 Bayes net  $B$  = naive Bayes learning on TrainingSet;
12 while NumAxioms  $\leq$  MaxAxioms or CPULimit  $\leq$  MaxCPU do
13   forall unproved  $C \in \text{Conjectures}$  do
14     Evaluate  $B$  on  $\text{Symbols}(C) \cup \text{CMOSet}(C)$  to order  $\text{Axioms}(C)$ ;
15     UseAxioms = NumAxioms most relevant axioms from  $\text{Axioms}(C)$ ;
16     if CPULimit < CSACPULimit then
17       while  $\exists S \in \text{CSASet}(C): \text{UseAxioms} \subseteq S$  do
18         Add next most relevant axiom to UseAxioms;
19       end
20     end
21     while  $\exists M \in \text{CMOSet}(C): \neg(\exists A \in \text{UseAxioms}: M \models \neg A)$  do
22       Add next most relevant  $A: M \models \neg A$ , to UseAxioms;
23     end
24     Result = Run systems on  $\text{UseAxioms} \models C$  with CPULimit;
25     ProcessResult(C, Result, UseAxioms);
26   end
27   Bayes net  $B$  = naive Bayes learning on TrainingSet;
28   if Any result is Theorem then
29     NumAxioms = MinAxioms, CPULimit = MinCPU;
30   else if NumAxioms < MaxAxioms then
31     NumAxioms =  $2 * \text{NumAxioms}$ ;
32   else CPULimit =  $4 * \text{CPULimit}$ ;
33 end
34 Procedure ProcessResult(C, Result, AxiomsTried):
35 if Status(Result) = Theorem then
36   TrainingSet = TrainingSet  $\cup \{\langle \text{Symbols}(C), \text{AxiomsInProof}(\text{Result}) \rangle\}$ ;
37 else if Status(Result) = CounterSatisfiable then
38    $\text{CSASet}(C) = \text{CSASet}(C) \cup \{\text{AxiomsTried}\}$ ;
39   forall  $M \in \text{ModelsFound}(\text{Result})$  do
40     forall  $F \in (\text{AllAxioms} \cup \text{Conjectures}): M \models \neg F$  do
41        $\text{CMOSet}(F) = \text{CMOSet}(F) \cup \{M\}$ ;
42       TrainingSet = TrainingSet  $\cup \{\langle M, F \rangle\}$ ;
43     end
44   end
45 end

```

Fig. 2. MaLAREa Algorithm

- 22; t144_relat_1; reflexivity_r1_tarski,t116_relat_1,t144_relat_1
- 23; t145_funct_1; d10_xboole_0,d1_relat_1,d3_relat_1,d3_tarski,
reflexivity_r1_tarski,t144_relat_1,t145_funct_1,t167_relat_1,t2_xboole_1,
t33_zfmisc_1,t88_relat_1,t99_relat_1
- 26; t146_relat_1; d10_xboole_0,reflexivity_r1_tarski,t144_relat_1,
t146_relat_1,t160_relat_1,t2_xboole_1,t44_relat_1,t45_relat_1
- 32; t160_relat_1; d10_xboole_0,dt_k5_relat_1,reflexivity_r1_tarski,
t144_relat_1,t160_relat_1,t2_xboole_1,t3_xboole_1,t44_relat_1

The model reasoner looked at the four suggested axioms, and saw that models 22 and 32 were not excluded by them, i.e., none of the four axioms were false in those two models of the negated conjecture, while model 23 was already excluded by axiom `t88_relat_1`, and model 26 by axiom `t45_relat_1`. The model reasoner therefore started to go through additional axioms in their order of relevance, and found that axiom `t116_relat_1` excluded model 22, and axiom `t44_relat_1` excluded model 32. At this point all known models of the negated conjecture were excluded, so this axiom selection was used for pass 3. The resultant problem was found to be countersatisfiable, yielding the following model (and this was the only new model of the negated conjecture `t144_relat_1` found in pass 3):

- 181; t144_relat_1; d3_tarski,reflexivity_r1_tarski,t144_relat_1

The relevance based selection suggested these four axioms for pass 4:

`t20_relat_1,t99_relat_1,t88_relat_1,t45_relat_1`

which is just a permutation (caused by relevance update) of the four initial axioms suggested in pass 3. Since the previous problem (containing two additional axioms) was known (now) to be countersatisfiable, the subset precheck caused the addition of axiom `d3_tarski`, at which point the problem was not known to be countersatisfiable. There were two reasons for this axiom becoming very relevant at this point, and being the next in the relevance queue: The first reason was the “standard learning” argument. In pass 3, this axiom (definition of set-theoretical inclusion) was successfully used to prove several theorems, raising its overall relevance. The second reason was that this axiom became known to be false in a new model (181) of the negated conjecture, and this fact was taken into account by the relevance learner. After this addition by the the subset precheck, the model reasoner inspected the axiom selection. Since only one new model (181) was added in pass 3, and that model was already excluded by `d3_tarski` (which did not exclude any of the old models), it just proceeded to exclude the same two remaining models (22 and 32) by adding the same two axioms `t116_relat_1` and `t44_relat_1` axioms as before. In pass 4 this problem was found to be countersatisfiable, and three more models of the negated conjecture were produced:

- 315; t144_relat_1; t143_relat_1,t144_relat_1
- 316; t145_funct_1; d12_funct_1,d13_relat_1,d1_relat_1,t143_relat_1,
t144_relat_1,t145_funct_1,t146_relat_1,t147_funct_1,t33_zfmisc_1
- 324; t160_relat_1; d10_xboole_0,d13_relat_1,d1_relat_1,d3_relat_1,
d3_tarski,d4_relat_1,d5_relat_1,d8_relat_1,dt_k5_relat_1,
reflexivity_r1_tarski,t144_relat_1,t160_relat_1,t2_tarski,t33_zfmisc_1

The relevance based selection suggested these four axioms for pass 5:

```
t20_relat_1, d3_tarski, t99_relat_1, t143_relat_1
```

Note that `d3_tarski` and `t143_relat_1` replaced `t88_relat_1` and `t45_relat_1` in the top four positions of the relevance order. In case of `d3_tarski` this is again both because it was successfully used to prove some more other theorems in pass 4, and because it now excludes one more model (324) of the negated conjecture. In case of `t143_relat_1` it is solely because it excludes two fresh models (315, 316) of the negated conjecture (and especially the exclusion of the model 315 is a relatively strong relevance boost, because there are only two formulae with that feature, and this rarity is taken into account by the Bayes learner). This basic specification was not known to be countersatisfiable, so the model reasoner started to inspect it. The four initial axioms already excluded models 23, 181, 315, 316 and 324. Models 22, 26, and 32 were not excluded by any of these four axioms, so first the axiom `t45_relat_1` was added, excluding model 26, and then the axiom `reflexivity_r1_tarski` was (finally) sufficiently relevant and excluded the remaining models 22 and 32. This problem produced the proof, using axioms `d3_tarski`, `t143_relat_1`, and `t20_relat_1`.

It is interesting to note in the pass 3 that the axiom `reflexivity_r1_tarski` excluded all of the four models, but it was not sufficiently relevant to be selected. Similarly, in pass 4, `reflexivity_r1_tarski` excludes all of the models. This suggests that an alternative model covering algorithm could be useful for making the specifications smaller. Preliminary experiments suggest that using, e.g., MiniSat+, for finding the optimal model coverings would be feasible. It might also be worth experimenting with exchanging relevance-greed for covering-greed, i.e., taking the axioms that exclude most candidate models first.

5.1 Partial vs. Total Models

The reader might wonder why the model reasoner added the axioms `t45_relat_1` and `reflexivity_r1_tarski` (this one alone would be sufficient, as it also excludes model 26) in the last pass. These axioms were not needed for the final proof, i.e., in every model of the negated conjecture at least one of the necessary axioms (`d3_tarski`, `t143_relat_1`, and `t20_relat_1`) should be false. Why then don't they exclude models 22, 26, and 32? The answer is in the implementation of models. The models produced by Mace4 are limited to the interpretation of symbols that appear in the problem formulae. The axiom `d3_tarski` defines `subset` in terms of `in` (membership), but `in` did not appear in the problems that led to the construction of models 22, 26, and 32. Similarly for `t143_relat_1`. Therefore the truth values of these two axioms are undefined in the three models.

This points to another interesting choice: For efficient proving and model finding we want to constrain the relevant set of symbols as much as possible. Often this leads to success – the number of theorems that can be proved by exploring Boolean properties of set-theoretical union, inclusion, difference, and subset, without any word about membership, is a good example. On the other hand, the notion of “relevance” evolves as theorems are proved, and it may be necessary expand some of the definitions, as happened in the proof of `t144_relat_1`.

Such expansion, however, makes some of the earlier model-based information inaccurate in some sense.

There are several ways to cope with these problems. The current solution is just to ignore it: the worst that can happen is that extra axioms are added by the model reasoning. One (quite feasible) alternative is to try to extend all models to the set of “relevant symbols”, defined as the union of symbols appearing in the axioms that are above some relevance threshold, or the union of symbols appearing in all relevant models, etc. The model extension could be done by adding tautologies containing the new symbols, or by manually adding some default interpretation of the new symbols to the existing models, providing a default way for extending partial models to total models. The danger of this approach is that such total models are mostly interesting on only their partial symbol domain, and evaluation of many formulae outside that domain will flood the system with a lot of boring values. Another interesting approach could be to watch the relevance of models in a way similar to watching the relevance of axioms, and preferring evaluation in more relevant models. This option is likely to be used anyway, if the number of generated models gets too high, and evaluation of all formulae in them too expensive.

6 Other Performance Related Additions to MaLAREa

6.1 Term Structure

It is not difficult to come up with examples when just symbol-based similarity can be too crude. One way to picture this is to realize that all the symbols used in the MPTP Challenge problems are introduced by their set-theoretical definitions that can recursively be expanded to just the basic set theory level, i.e., to formulae containing only the $\in$ and $=$ symbols. If that were done, symbol-based similarity would become completely useless, but the expanded formulae would still be more or less close to one another when considering the structure of terms and subformulae. This suggests that structural similarity is likely to be a useful complement to symbol-based similarity. This is confirmed by the experimental results shown in Section 7. The practical implementation in MaLAREa takes an advantage of the shared term mechanisms used in the E prover. E prover keeps only one copy of each term in its shared term bank, and it also treats all formulae for clausification purposes internally as shared terms. The shared terms have internal serial numbering, used, e.g., for compact printing. Given these nice functionalities, it was straightforward to write a small E-based program that reads all formulae appearing in all problems into a shared term bank, and then for each formula it prints the internal numbers of all the shared terms (i.e., also subformulae) appearing in it. The information is printed in the following form:

```
terms(t144_relat_1,[492,491,8325,8324,1803,2153]).
```

There are about 10000 shared terms in the MPTP Challenge formulae (compared to 418 symbols), giving a much finer similarity measure that can be used for learning in the same way as the symbol features. One additional modification was implemented to accommodate the problem of different (normalized)

variables appearing in the shared terms, making terms that differ only by variable renaming have different numbers. This was solved by renaming all variables in each formula to just one variable before handing the formulae to the shared term extractor. MaLAREa has options for specifying all these similarity measures (symbols, variable-renamed shared terms, standard shared terms), and their combinations.

6.2 ATP Systems

The Vampire [14] system was previously not usable in MaLAREa, because its proof output was hard to parse. Therefore TPTP format output was added to Vampire, allowing its use as an ATP backend along with E and SPASS. First experiments on the MPTP Challenge show that using Vampire does not help the combination of E and SPASS. One reason can be traced back to Vampire’s tuning on the TPTP library: the Vampire experiment (using Vampire with a different classifier) significantly improves Vampire’s performance on the MPTP Challenge problems. This is caused by a different (much better for MPTP) set of strategies being used by Vampire for CNF problems than the strategies used by Vampire for FOF problems. This again suggests that addition of ATP strategy learning to MaLAREa could be quite useful. Such effort would be greatly facilitated by having a common TPTP format for ATP parameters, and systems like E, SPASS, and Vampire actually using that format.

Other work that had positive impact was the correction of a bug in the proof mode of the E prover. This error had caused the strategies used in the fast “assurance” mode to be replaced by dummy strategies in proof mode, often causing failure in the proof mode. This in turn prevented learning the axiom relevance from some of the theorems proved by E. This correction improved MaLAREa’s performance on the MPTP Challenge benchmark by 4 problems.

7 Results

The set of problems from the chainy division of the MPTP Challenge was used for evaluation. This set of problems was created with the intention of providing a benchmark for large theory reasoning. It consists of 252 related mathematical problems, translated by the MPTP system [22] from the Mizar library. The conjectures of the problems are the Mizar theorems that are recursively needed for the Mizar proof of one half (one of two implications) of the general topological Bolzano-Weierstrass theorem. The whole problem set contains 1234 formulae and 418 symbols. The problems in the chainy division intentionally contain all the “previous knowledge” as axioms. This results in an average problem size of ca. 400 formulae. The challenge has an overall time limit policy, i.e., instead of solving problems one-at-a-time with a fixed time limit, the overall time limit of 21 hours can be used in any way for solving the problems. The experiments on this set of problems were done on 2.33GHz Intel Xeons (4MB cache) with 1GB RAM. These machines seem to be about 4% faster than the machines used for evaluating many other system for the MPTP Challenge in 2007.

The first experiment corresponds to the standard MPTP Challenge, i.e., solving as many problems as possible within the overall time limit. Table 1 shows the results of three different runs: MaLAREa in the standard setting presented in Section 2⁴, MaLAREa with the term structure learning presented in Section 6.1, and finally MaLAREa with term structure learning and the semantic guidance presented in Section 4. All three runs used the default minimal and maximal time and axiom limits (1-64s, 4-256 axioms, 1s threshold for the countersatisfiability precheck). All of them were left to run beyond the MPTP Challenge overall time limit of 21 hours, until they stopped themselves by reaching the maximal axiom and time limits.

Table 1. Full run of three versions of MaLAREa on the MPTP Challenge

description	total solved	solved in 21 hours	passes in 21 hours	total passes	last pass with success	time to stop (hours)	time to solve last
standard	149	144	114	234	208	83	66
with TS	154	149	148	263	237	55	47
with TS & SG	165	161	121	195	170	52	37

The table shows that the addition of term structure improves the original version by 5 problems (both within the 21 hour time limit and overall), and that the semantic guidance then contributes an additional 12 (within the 21 hour time limit), resp. 11 (without time limit) solved problems. Although the numbers of solved problems differ quite significantly, it is also interesting to note that the version with semantic guidance peaked sooner (37 hours to solve all of the 165 problems) than the other versions. This faster convergence becomes even more apparent when we consider just the number of problems solved in the fast 1s passes following the all-axioms 64s first pass. This first pass obviously gave the same values for all three versions (because it uses all the axioms), solving 100 problems, all within 1s. However the numbers of problems solved in the following 1s passes differ quite considerably – see Table 2. This table shows that the version with semantic guidance solved more problems (56) in the following 1s passes than the version without semantic guidance solved in all following passes (154 problems solved - 100 solved in the first pass = 54). It should also be noted that the addition of Paradox typically makes these low-time limit runs faster than in the version without semantic guidance, because problems with few axioms are often countersatisfiable, and Paradox is much better at detecting that than E and SPASS.

Given this good performance with low time limits, it is natural to ask what happens when the time limit is reduced to 1s in the all-axioms first pass. This has a quite pragmatic motivation: both E and SPASS take the full 64s on the 152 problems not solved in the first pass (252 problems, 100 solved in the first pass),

⁴ But with visible improvement caused by correction of the E bug. Even without this correction, standard MaLAREa was the best performing system on the MPTP challenge at the time of the July 2007 evaluation.

Table 2. Statistics of the Initial 1s Passes

description	total solved	solved in 1st pass	solved in 1s passes	number of 1s passes
standard	149	100	36	36
with TS	154	100	41	56
with SG,TS	165	100	56	54

Table 3. Statistics of the 1s-only passes of the version using TS and SG

pass	ax limit	solved	cumul	pass	ax limit	solved	cumul	pass	ax limit	solved	cumul
1	0	60	60	12	4	3	120	23	4	0	134
2	256	7	67	13	4	1	121	24	8	1	135
3	256	1	68	14	4	2	123	25	4	0	135
4	4	12	80	15	4	3	126	26	8	0	135
5	4	10	90	16	4	0	126	27	16	0	135
6	4	14	104	17	8	2	128	28	32	4	139
7	4	5	109	18	4	2	130	29	4	0	139
8	4	2	111	19	4	0	130	30	8	0	139
9	4	3	114	20	8	0	130	31	16	0	139
10	4	2	116	21	16	3	133	32	32	0	139
11	4	1	117	22	4	1	134	33	64	2	141

so the pass takes more than 5 hours. The second experiment thus imposed a time limit of 1s in the all-axioms first pass. The results are shown in Table 3. Every pass takes 5-10 minutes (depending on how much Paradox is successful, and if (and how quickly) Mace produces a model). The value of 128 solved problems (more than 50%), which is beyond what any other ATP system solves in 21 hours, is reached in about 2-3 hours. The value of 104 solved problems, which is above the combined forces of the two underlying ATPs run both with 64s time limit, can be reached in six passes, i.e., in ca. 30-60 minutes.

8 Future Work and Conclusion

Human mathematics is very often a large theory. Humans are both good at developing large theories, and at reasoning within them. These two capabilities are certainly connected: it takes a lot of theory development to prove major mathematical results like Fermat’s last theorem and the Poincare conjecture. The system presented here tries to reason only in a fixed theory. It would be interesting to add theory development to the system. The first try could be just addition of useful lemmas created during the proof attempts. Another line of research is work on the semantic part. Human mathematicians have a well developed store of interesting examples and counterexamples that they use for testing their theories, suggesting conjectures, and getting their proofs right. The models created so far are simple – finite, often of small cardinality. It would be interesting to try to accommodate infinite domains, by generalizing the current simple model reasoning to perhaps less precise, but more sophisticated evaluation algorithms that are probably used by humans.

A lot of work on the system can be done by improving the existing components. Different learning algorithms (for different parts of the system) can be

experimented with, different backends can be used in different settings (see, e.g., the note about strategy learning in Section 6.2), and the usage of various components in the overall loop can certainly be experimented with much more. Another challenge is to accommodate large theories that are not so strictly consistent in symbol and axiom naming as the MPTP problems, e.g., by using even more abstract structural information. While many of these extensions are interesting AI research, it is also good to keep in mind that the current performance of the system already makes it useful, e.g., for proof assistance, and to further work on its practical usability by e.g., the Mizar and Isabelle formalizers.

Acknowledgements. Thanks to Bill McCune for his help with various Mace4 and clausefilter issues, and to Stephan Schulz and Christoph Weidenbach for correcting the E and SPASS bugs that influenced the performance of the system.

References

1. Baumgartner, P., Fuchs, A., Tinelli, C.: Darwin - A Theorem Prover for the Model Evolution Calculus. In: Sutcliffe, G., Schulz, S., Tammet, T. (eds.) *Proceedings of the Workshop on Empirically Successful First Order Reasoning, 2nd International Joint Conference on Automated Reasoning* (2004)
2. Carlson, A., Cumby, C., Rosen, J., Roth, D.: SNoW User's Guide. Technical Report UIUC-DCS-R-99-210, University of Illinois, Urbana-Champaign (1999)
3. Claessen, K., Sorensson, N.: New Techniques that Improve MACE-style Finite Model Finding. In: Baumgartner, P., Fermueller, C. (eds.) *Proceedings of the CADE-19 Workshop: Model Computation - Principles, Algorithms, Applications* (2003)
4. Denney, E., Fischer, B., Schumann, J.: Using Automated Theorem Provers to Certify Auto-generated Aerospace Software. In: Basin, D., Rusinowitch, M. (eds.) *IJ-CAR 2004. LNCS (LNAI)*, vol. 3097, pp. 198–212. Springer, Heidelberg (2004)
5. Fuchs, M.: Controlled Use of Clausal Lemmas in Connection Tableau Calculi. *Journal of Symbolic Computation* 29(2), 299–341 (2000)
6. McCune, W.W.: Mace4 Reference Manual and Guide. Technical Report ANL/MCS-TM-264, Argonne National Laboratory, Argonne, USA (2003)
7. Meng, J., Paulson, L.: Lightweight Relevance Filtering for Machine-Generated Resolution Problems. In: Sutcliffe, G., Schmidt, R., Schulz, S. (eds.) *Proceedings of the FLoC 2006 Workshop on Empirically Successful Computerized Reasoning, 3rd International Joint Conference on Automated Reasoning. CEUR Workshop Proceedings*, vol. 192, pp. 53–69 (2006)
8. Meng, J., Paulson, L.: Translating Higher-order Problems to First-order Clauses. *Journal of Automated Reasoning* 40(1), 35–60 (2008)
9. Plaisted, D.A., Yahya, A.: A Relevance Restriction Strategy for Automated Deduction. *Artificial Intelligence* 144(1-2), 59–93 (2003)
10. Pudlak, P.: Search for Faster and Shorter Proofs using Machine Generated lemmas. In: Sutcliffe, G., Schmidt, R., Schulz, S. (eds.) *Proceedings of the FLoC 2006 Workshop on Empirically Successful Computerized Reasoning, 3rd International Joint Conference on Automated Reasoning. CEUR Workshop Proceedings*, vol. 192, pp. 34–52 (2006)
11. Pudlak, P.: Draft Thesis: Verification of Mathematical Proofs (2007)

12. Quaife, A.: *Automated Development of Fundamental Mathematical Theories*. Kluwer Academic Publishers, Dordrecht (1992)
13. Ramachandran, D., Reagan, P., Goolsbey, K.: First-orderized ResearchCyc: Expressiveness and Efficiency in a Common Sense Knowledge Base. In: Shvaiko, P. (ed.) *Proceedings of the Workshop on Contexts and Ontologies: Theory, Practice and Applications* (2005)
14. Riazanov, A., Voronkov, A.: The Design and Implementation of Vampire. *AI Communications* 15(2-3), 91–110 (2002)
15. Schulz, S.: E: A Brainiac Theorem Prover. *AI Communications* 15(2-3), 111–126 (2002)
16. Sutcliffe, G.: The 3rd IJCAR Automated Theorem Proving Competition. *AI Communications* 20(2), 117–126 (2007)
17. Sutcliffe, G., Dvorsky, A.: Proving Harder Theorems by Axiom Reduction. In: Russell, I., Haller, S. (eds.) *Proceedings of the 16th International FLAIRS Conference*, pp. 108–112. AAAI Press, Menlo Park (2003)
18. Sutcliffe, G., Puzis, Y.: SRASS - a Semantic Relevance Axiom Selection System. In: Pfenning, F. (ed.) *CADE 2007. LNCS (LNAI)*, vol. 4603, pp. 295–310. Springer, Heidelberg (2007)
19. Sutcliffe, G., Suttner, C.B.: The TPTP Problem Library: CNF Release v1.2.1. *Journal of Automated Reasoning* 21(2), 177–203 (1998)
20. Sutcliffe, G., Zimmer, J., Schulz, S.: TSTP Data-Exchange Formats for Automated Theorem Proving Tools. In: Zhang, W., Sorge, V. (eds.) *Distributed Constraint Problem Solving and Reasoning in Multi-Agent Systems. Frontiers in Artificial Intelligence and Applications*, vol. 112, pp. 201–215. IOS Press, Amsterdam (2004)
21. Urban, J.: MizarMode - An Integrated Proof Assistance Tool for the Mizar Way of Formalizing Mathematics. *Journal of Applied Logic* 4(4), 414–427 (2006)
22. Urban, J.: MPTP 0.2: Design, Implementation, and Initial Experiments. *Journal of Automated Reasoning* 37(1-2), 21–43 (2006)
23. Urban, J.: MaLARea: a Metasystem for Automated Reasoning in Large Theories. In: Urban, J., Sutcliffe, G., Schulz, S. (eds.) *Proceedings of the CADE-21 Workshop on Empirically Successful Automated Reasoning in Large Theories*, pp. 45–58 (2007)
24. Veroff, R.: Using Hints to Increase the Effectiveness of an Automated Reasoning Program: Case Studies. *Journal of Automated Reasoning* 16(3), 223–239 (1996)
25. Weidenbach, C.: Combining Superposition, Sorts and Splitting. In: Robinson, A., Voronkov, A. (eds.) *Handbook of Automated Reasoning*, pp. 1965–2011. Elsevier Science, Amsterdam (2001)
26. Zhang, Y., Sutcliffe, G.: Lemma Management Techniques for Automated Theorem Proving. In: Konev, B., Schulz, S. (eds.) *Proceedings of the 5th International Workshop on the Implementation of Logics*, pp. 87–94 (2005)